QATIP AWS

Lab 6 Jenkins Terraform Pipelining (VSC)

Lab objectives 1
Where the work runs 2
Before you begin 2
Prepare VS Code and PowerShell 2
Task 1 Create the Jenkins EC2 instance 3
Task 2 Create the S3 state bucket 4
Task 3 Configure Jenkins 4
Task 4 Prepare the GitHub repository 4
Task 5 Add AWS credentials to Jenkins 5
Task 6 Configure the deployment pipeline 6
Task 7 Verify the run 6
Task 8 Change and trigger the pipeline 7
Task 9 Configure and run the destroy pipeline 7
Final cleanup 8
Troubleshooting 9
Notes and references 10

Windows VS Code and PowerShell edition

QATIP AWS • Optional lab • Region us-west-2 Oregon

This lab uses Jenkins to deploy, change and destroy an AWS S3 bucket with Terraform. You prepare the lab files and run the provisioning scripts from VS Code on Windows using PowerShell. Jenkins remains on an Ubuntu EC2 instance and runs the Terraform pipeline there.

Lab objectives

- Create a dedicated VPC, public subnet and EC2 instance with Jenkins installed.
- Create an S3 bucket for remote Terraform state.
- Configure Jenkins to obtain its Terraform configuration from GitHub.
- Use Jenkins credentials to create, replace and destroy a test bucket.
- Explore a manual pipeline run and, if time permits, a GitHub webhook.

Where the work runs

Location	Purpose
Windows workstation	VS Code editing, AWS CLI setup, PowerShell scripts and SSH access
Ubuntu EC2 in us-west-2	Jenkins service, Git checkout and Terraform commands
GitHub repository	main.tf, Jenkinsfile and destroy/Jenkinsfile
S3 in us-west-2	Separate state bucket and Terraform managed test bucket

The Jenkinsfiles deliberately use `sh`: those commands run on the Linux Jenkins server. You do not run them in the Windows terminal. Cloud9, WSL, Git Bash and a local Jenkins installation are not required.

Before you begin

Complete Lab 0. Your Windows machine needs VS Code, PowerShell 7.4 or later, AWS CLI v2 and the Windows OpenSSH client. GitHub access and a lab account with the required EC2, VPC, S3 and SSM permissions are also required. A local Terraform installation is optional because Jenkins runs Terraform.

All AWS resources in this lab use us-west-2. Do not upload real data to the test bucket.

Prepare VS Code and PowerShell

1. In VS Code choose **File > Open Folder**, select `C:\awslabs\06`, and trust the folder after reviewing its contents.

2. Open **Terminal > Select Default Profile**, select **PowerShell** for PowerShell 7, then open a new terminal. Check the version; Windows PowerShell 5.1 cannot run these scripts.

```
$PSVersionTable.PSVersion
```

If the result is below 7.4, install PowerShell 7 using the following command in your existing terminal. Reopen VS Code afterwards and select the new PowerShell profile. If winget is unavailable, ask the instructor to install PowerShell from Microsoft's official download page.

```
winget install --id Microsoft.PowerShell --exact --source winget
```

For an existing PowerShell 7 installation, use:

```
winget upgrade --id Microsoft.PowerShell --exact --source winget
```

Reopening VS Code is sufficient for a normal side by side installation; this lab does not require a planned Windows reboot.

3. Run these commands in the new PowerShell terminal, one line at a time.

```
Set-Location C:\awslabs\06
Get-ChildItem -Recurse -Filter *.ps1 | Unblock-File
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
aws --version
ssh -V
```

The execution policy change applies only to this terminal and does not override an organisation's Group Policy. If `aws` or `ssh` is missing, install the approved AWS CLI v2 or Windows OpenSSH client before proceeding. Reopen the terminal after changing PATH.

4. Use the AWS credentials or sign-in method supplied in Lab 0. Select your lab profile if required, then verify the active account.

```
aws sts get-caller-identity --region us-west-2
```

If you use a named profile, set it first, for example `$env:AWS_PROFILE = 'training'`. Check that the account and identity match the lab portal. Do not continue with a personal or unrelated AWS account.

Task 1 Create the Jenkins EC2 instance

1. In VS Code review Initialize-Jenkins.ps1, Lab.Common.ps1, jenkins-user-data.sh and Remove-Jenkins.ps1.

The setup script creates a dedicated network and an Ubuntu Server 24.04 EC2 instance in us-west-2. It installs Java 21, Jenkins LTS, Git and Terraform on that instance. The `.sh` file is EC2 user data: the script sends it to Ubuntu automatically. You do not execute it on Windows.

The script resolves the Ubuntu AMI from Canonical's regional SSM parameter, gives the resources a unique lab identifier, and records their IDs in `.local\jenkins-lab.json`. It uses a t3.small instance with Standard CPU credits. Keep the `.local` folder until cleanup is complete; it contains the deployment record and private SSH key. Never upload it to GitHub.

2. Start the deployment in the VS Code PowerShell terminal.

```
.\Initialize-Jenkins.ps1
```

SSH port 22 and Jenkins port 8080 are initially limited to your detected public IPv4 address. If automatic IP detection is blocked, use your approved public IPv4 address, replacing this documentation example:

```
.\Initialize-Jenkins.ps1 -ClientCidr '203.0.113.10/32'
```

If the instructor specifies a different allowed availability zone in Oregon, supply it as follows:

```
.\Initialize-Jenkins.ps1 -AvailabilityZone 'us-west-2b'
```

If SSM access is blocked, the instructor can supply an approved Canonical Ubuntu 24.04 x86_64 AMI from us-west-2 using the `-AmiId` parameter. Do not change region to resolve an error.

3. When EC2 is running, the script displays the Jenkins URL. Installation is still progressing on the server. Retrieve the initial administrator password with:

```
.\Get-JenkinsAccess.ps1
```

This retries while the server starts and installs its software. Allow several minutes. Keep the displayed password private. Record the URL and instance ID in your lab notes; use a password manager for passwords.

Checkpoint: the script displays a URL such as `http://PUBLIC_IP:8080` and an initial administrator password. If setup fails, keep `.local` and use the cleanup instructions before creating another deployment.

Discuss: why is the Jenkins service created outside the Terraform pipeline that it will run? What would happen if the pipeline destroyed its own execution host?

Task 2 Create the S3 state bucket

1. Open the AWS Console and confirm the active lab account. Open S3 and choose **Create bucket**.

2. Select **General purpose** and **US West Oregon us-west-2**. Use a globally unique name such as jenkins-state-yourname-123456 in lowercase. Record the exact name as your state bucket.

3. Keep **Block all public access** enabled and **ACLs disabled**. Enable **Bucket Versioning** to retain earlier state versions. Keep the default server side encryption with Amazon S3 managed keys and create the bucket.

4. Verify the bucket's region is us-west-2.

This bucket stores Terraform state. It is created before the pipeline and is not managed by the example Terraform configuration. The test bucket created later must have a different name. Leave the state bucket in place until the destroy pipeline has completed.

Task 3 Configure Jenkins

1. Open the URL displayed by the script in a browser: http://PUBLIC_IP:8080.

2. On **Unlock Jenkins**, enter the initial administrator password retrieved in Task 1 and select **Continue**.

3. Select **Install suggested plugins**. Wait for installation to finish. The pipelines need Pipeline, Git and Credentials Binding; the optional webhook exercise also needs the GitHub plugin.

4. Create an administrator user when prompted and keep its password securely. If the wizard offers **Skip and continue as admin**, that lab shortcut is also acceptable; retain the initial administrator password for subsequent sign-in.

5. Confirm the Jenkins URL, choose **Save and Finish**, then **Start using Jenkins**.

6. In **Manage Jenkins > Nodes**, check that the built-in node is online and has at least one executor. This small lab runs the pipeline on that Ubuntu node.

Checkpoint: the Jenkins dashboard loads and you can create a new item.

Lab use only: this example uses HTTP and builds on the Jenkins controller to keep the exercise small. Use only disposable lab credentials. A production design needs HTTPS, controlled access, dedicated build agents and an appropriate AWS identity design. Remove the lab when finished.

Task 4 Prepare the GitHub repository

Use your existing GitHub account or register for a free account at https://github.com. Follow the current GitHub prompts. Keep your login password outside the lab files.

1. Create a repository with a name such as jenkins-terraform-lab. Select **Public** and add a README. Confirm that the default branch is main. A public repository lets Jenkins read these non-secret files without an extra GitHub credential.

2. In VS Code open the supplied main.tf. Replace REPLACE_WITH_STATE_BUCKET with the exact state bucket name from Task 2.

3. Replace REPLACE_WITH_TEST_BUCKET with a different globally unique, lowercase name, such as jenkins-test-yourname-123456. Do not use the state bucket name.

4. Save main.tf. Check that the provider and backend region both remain us-west-2. The state key is jenkins-lab/terraform.tfstate and S3 lockfile support is enabled.

5. In GitHub choose **Add file > Upload files**. Upload your edited main.tf and the supplied root Jenkinsfile from C:\awslabs\06. Commit the changes to main.

6. Open each uploaded file and confirm its name and contents. Record the repository's HTTPS URL, for example https://github.com/YOUR-USER/jenkins-terraform-lab.git.

The Jenkinsfile uses checkout scm, so you do not need to edit a repository URL inside it. Task 6 supplies that URL through the Jenkins job configuration.

Upload only the specified files. Do not upload the PowerShell provisioning scripts, the .local folder, PEM keys, AWS credentials, Terraform state or plan files. The supplied .gitignore is available if you later use Git locally, but GitHub browser uploads do not apply its rules for you.

For this lab, edit files locally in VS Code and use GitHub's upload facility to commit each revision. Uploading an existing filename updates it. No files need to be downloaded from a remote IDE.

Checkpoint: the repository's main branch contains README.md, main.tf and Jenkinsfile. Neither replacement marker remains in main.tf.

Task 5 Add AWS credentials to Jenkins

The Windows AWS CLI and Jenkins run in different environments. Your local AWS sign-in lets the PowerShell script create the Jenkins host. It is not automatically copied into Jenkins. This task supplies the AWS credentials that Jenkins uses for Terraform, including access to the state bucket.

1. In Jenkins open Manage Jenkins > Credentials.

2. Open **System**, then **Global credentials unrestricted**, and choose **Add Credentials**. Menu wording may vary slightly by Jenkins version.

3. Select Username with password and Global scope.

4. Enter the lab user's **Access key ID** as the username and **Secret access key** as the password, using the values supplied by your lab portal.

5. Set **ID** to aws_user exactly. Add a description such as AWS lab credentials, then select **Create**.

The Jenkinsfiles bind this entry to AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY only around the Terraform commands. Keep credentials out of GitHub, main.tf and the Jenkinsfiles. Do not use AWS root access keys.

If the lab portal supplies a session token

Temporary AWS credentials require all three values. If the portal supplies a session token as well as the access key and secret, add a second Jenkins credential:

1. Select **Add Credentials**, choose **Secret text**, and paste the session token into **Secret**.

2. Set **ID** to aws_session_token exactly and save it.

3. When running either pipeline, enable the supplied USE_SESSION_TOKEN build parameter. Jenkins may expose parameters after the first run has loaded the Jenkinsfile. If that first run fails for temporary credentials, start another run with **Build with Parameters** and enable the option.

For a normal two-value access key, leave USE_SESSION_TOKEN unchecked and do not create the extra entry. If temporary credentials expire, update both Jenkins entries with a matching fresh set before retrying. A browser-only AWS sign-in cannot substitute for these values; ask the instructor if the portal does not supply API credentials.

Permissions: the Jenkins identity needs access to create, read, update and delete the test bucket, and to list the state bucket and read/write its state object. S3 locking also requires read, write and delete access to jenkins-lab/terraform.tfstate.tflock. Follow the lab's supplied permissions; do not broaden access in response to an unexplained denial.

Task 6 Configure the deployment pipeline

1. From the Jenkins dashboard choose **New Item**. Enter Terraform Pipeline, select **Pipeline**, then choose **OK**.

2. In Pipeline, set Definition to Pipeline script from SCM and SCM to Git.

3. Enter your repository's HTTPS URL. Leave Git credentials unset for this public repository.

4. Set **Branch Specifier** to */main and **Script Path** to Jenkinsfile. Save the job.

5. Choose **Build Now**. If the job already shows **Build with Parameters**, choose the correct USE_SESSION_TOKEN value and start the build. Follow the Task 5 note if a first run is needed to expose this parameter.

Jenkins checks out the repository on Ubuntu, checks the installed Terraform version, initialises the backend, validates the configuration, saves a plan and applies that saved plan. Terraform is installed during EC2 bootstrap, so the pipeline does not download it again or need sudo permissions.

Task 7 Verify the run

1. Select the build number, then open **Console Output** or the available pipeline stage view.

2. Verify that **Checkout**, **Check Terraform**, **Init**, **Validate**, **Plan** and **Apply** succeed. The first deployment should create one test bucket.

3. In the AWS Console open S3 and locate the test bucket named in main.tf. Confirm that its region is us-west-2.

4. Open the separate state bucket. Verify that jenkins-lab/terraform.tfstate exists. A .tflock object may appear while Terraform holds the lock; it should be removed after a completed operation.

5. Explore the build history, stage timing and console output. Return to the dashboard.

Checkpoint: the build succeeds, the test bucket exists, and the state object is in the state bucket. The backend bucket itself is not part of the Terraform plan.

Both pipelines remove their local Jenkins workspace after each run. The next run obtains the configuration from GitHub and the current state from S3. Each run resolves an AWS provider compatible with the constraint in main.tf; a maintained production repository would also commit a reviewed dependency lock file.

Task 8 Change and trigger the pipeline

Run a change manually

1. In VS Code open your local main.tf. Change the test bucket name by adding a few lowercase letters or digits, keeping it globally unique. Leave the backend bucket and state key unchanged.

2. Save the file, then upload the revised main.tf to the same GitHub repository and commit the change to main.

3. In Jenkins run Terraform Pipeline again, retaining the session token parameter setting if applicable.

4. Examine the plan and apply logs. A bucket name cannot be changed in place, so Terraform replaces the test bucket. Verify in S3 that the old empty test bucket has gone and the new one exists.

Optional GitHub webhook

A webhook must reach Jenkins from GitHub's servers. Your workstation-only security group rules do not allow that traffic.

1. From the local PowerShell terminal run:

```
.\Set-GitHubWebhookAccess.ps1 -Enable
```

This adds inbound TCP 8080 rules for GitHub's published IPv4 webhook ranges. If lab policy or the security group quota blocks the change, continue with manual builds.

2. If using temporary credentials, change defaultValue: false to defaultValue: true for USE_SESSION_TOKEN in the root Jenkinsfile, upload and commit it, and run one successful manual build with the option enabled. This loads the correct default for webhook builds, which cannot ask you to select parameters. For two-value access keys, leave the default false.

3. In the deployment job's **Configure > Build Triggers**, select **GitHub hook trigger for GITScm polling** and save. Do not enable this on the destroy job.

4. In GitHub open the repository's **Settings > Webhooks > Add webhook**. Use http://PUBLIC_IP:8080/github-webhook/ as the payload URL and application/json as the content type. Subscribe only to push events and leave the webhook active. This temporary lab uses HTTP; the SSL verification control has no effect on an HTTP URL. Do not disable SSL verification for HTTPS endpoints.

5. Change the test bucket name again locally, upload and commit main.tf, then inspect GitHub's **Recent Deliveries** and the Jenkins build history. Expect a successful delivery and an automatically started pipeline.

This HTTP webhook is a disposable training shortcut. A production webhook needs HTTPS and validation of a shared webhook secret. Before the destruction task, disable or delete the webhook and run:

```
.\Set-GitHubWebhookAccess.ps1 -Disable
```

Task 9 Configure and run the destroy pipeline

Do not skip resource destruction when time is short. If you cannot complete this pipeline exercise, use the instructor's assisted cleanup route on the next page.

1. Disable or delete the GitHub webhook if configured. Remove its ingress rules as described in Task 8. Wait for the deployment job to finish, then disable Terraform Pipeline in Jenkins so that it cannot recreate the bucket during cleanup.

2. In GitHub choose **Add file > Create new file**. Enter destroy/Jenkinsfile as the filename. In VS Code open the supplied C:\awslabs\06\destroy\Jenkinsfile, copy its entire contents into GitHub and commit to main.

3. Back in Jenkins choose **New Item**, enter Terraform Pipeline Destroy, select **Pipeline**, then choose **OK**.

4. Set **Definition** to **Pipeline script from SCM**, **SCM** to **Git**, the repository URL to your HTTPS URL, **Branch Specifier** to */main, and **Script Path** to destroy/Jenkinsfile. Save the job. Leave automatic build triggers disabled.

5. Run the destroy job. If using temporary credentials, enable USE_SESSION_TOKEN, following the Task 5 first-run note if necessary.

6. Read the **Destroy plan** stage in the console. It should target the Terraform managed test bucket only. The job then waits at **Approve destruction**.

7. Open the waiting input prompt from the build page or console and choose **Yes destroy** only after reviewing the plan. Depending on the interface, Jenkins may display the button as **Yes, destroy**.

8. Verify the **Apply destruction** stage succeeds and the test bucket disappears from S3. The Jenkins host and state bucket should still exist.

The supplied destroy pipeline initialises the same S3 backend and applies a saved destroy plan after your approval. Its filename is destroy/Jenkinsfile, with no .txt extension. Do not change the state bucket or key to make a destroy error disappear.

Checkpoint: the managed test bucket is absent and the state bucket still contains the final state. There must be no deployment job running while you perform final cleanup.

Final cleanup

Complete these steps even if you skipped the optional webhook task. Keep the state bucket until the managed test bucket has been destroyed successfully.

1. Confirm both pipeline jobs are idle and the deployment job is disabled. Disable or delete any GitHub webhook.

2. In the local VS Code PowerShell terminal preview the Jenkins cleanup:

```
.\Remove-Jenkins.ps1 -WhatIf
```

3. Run the cleanup and confirm the displayed lab identifier and account:

```
.\Remove-Jenkins.ps1
```

The script checks the active AWS account and resource ownership tags, terminates the recorded Jenkins instance, waits for termination and removes its security group, route table, subnet, internet gateway, VPC and AWS key pair. It removes the local private key and archives the deployment record. It does not delete S3 buckets or search for unrelated resources by name. If cleanup stops, retain .local, resolve the reported cause and retry; do not discard the deployment record.

4. In S3 verify that the Terraform test bucket has gone. Open the exact state bucket you created in Task 2. Choose **Empty** and follow the confirmation prompts to remove its objects, versions and delete markers. Versioning means deleting only the current state object is insufficient. Then delete the empty state bucket.

5. Verify that the tagged Jenkins EC2 instance is terminated and its dedicated VPC has been removed in us-west-2. Delete the GitHub repository at your discretion.

Instructor assisted cleanup if the destroy job cannot run

While the state bucket still exists, use the latest `main.tf` from the repository in a separate local working folder, with local Terraform 1.10 or later and the correct Lab 0 credentials. Do not run a competing Jenkins deployment. Run these commands, inspecting the proposed target and accepting the destruction prompt only for the lab test bucket:

```
terraform init
terraform plan -destroy
terraform destroy
```

Then perform the Jenkins and state-bucket cleanup above.

The test bucket must remain empty for normal Terraform destruction. If someone added objects, remove only those lab objects first. If your session expired, refresh the correct credentials before retrying; do not delete remote state to bypass an authentication or locking error.

Troubleshooting

PowerShell reports version or script policy errors

Run `$PSVersionTable.PSVersion` and use PowerShell 7.4 or later. Select the correct terminal profile in VS Code. Unblock the extracted scripts and use the process-scoped policy command on page 2. Ask the instructor if an organisation's policy prevents execution.

Jenkins cannot be reached or password retrieval keeps waiting

Check EC2 status and the public IP. A proxy, VPN or changing egress IP can make the detected IPv4 address differ from the address actually used for SSH or browser access. Ask the instructor to correct the security group's /32 source rules for ports 22 and 8080. Confirm your network allows outbound TCP 22 and 8080.

To inspect bootstrap without relaunching, first run these local PowerShell commands:

```
$lab = Get-Content .\.local\jenkins-lab.json -Raw | ConvertFrom-Json
$key = ".\.local\$($lab.KeyName).pem"
ssh -i $key "ubuntu@$($lab.PublicIp)"
```

At the Ubuntu prompt, these are Linux diagnostics, not local PowerShell commands:

```
sudo tail -n 80 /var/log/cloud-init-output.log
sudo systemctl status jenkins --no-pager
sudo journalctl -u jenkins -n 50 --no-pager
terraform version
exit
```

AWS reports AccessDenied or no available capacity

Verify the active account, us-west-2 region, lab permissions, quotas and permitted availability zones. Supply an instructor-approved Ubuntu AMI with `-AmiId` if SSM is restricted. Clean up a partially created deployment before retrying. A region change is not part of this lab.

Pipeline fails with credentials, state or repository errors

Check the `aws_user` credential ID, matching key pair and any required session token. Refresh expired credentials. Check the exact state bucket name, its us-west-2 region, state key and S3 lock permissions. Confirm the repository is public, its branch is `main`, and the selected script path exists. For a stale lock, ensure no Terraform run is still active before asking the instructor to investigate; do not delete a lock blindly.

Notes and references

Files supplied

File	Use
Initialize-Jenkins.ps1	Provision the Jenkins host and dedicated network from Windows
Get-JenkinsAccess.ps1	Retrieve the URL and initial administrator password
Remove-Jenkins.ps1	Remove recorded Jenkins resources with ownership checks
Set-GitHubWebhookAccess.ps1	Add or remove GitHub webhook source ranges
Lab.Common.ps1	Shared PowerShell support functions
jenkins-user-data.sh	Ubuntu bootstrap sent automatically to EC2
main.tf and Jenkinsfile	Terraform configuration and deployment pipeline
destroy/Jenkinsfile	Destruction pipeline with approval
.gitignore and Student_Guide.md	Local exclusions and copyable lab instructions

Official documentation

• Jenkins Linux installation: https://www.jenkins.io/doc/book/installing/linux/

• Terraform installation: https://developer.hashicorp.com/terraform/install

• Terraform S3 backend and locking: https://developer.hashicorp.com/terraform/language/backend/s3

• Ubuntu AMIs on AWS: https://ubuntu.com/aws/docs/aws-how-to/instances/find-ubuntu-images/

• AWS CLI file parameters: https://docs.aws.amazon.com/cli/latest/userguide/cli-usage-parameters-file.html

• GitHub webhook guidance: https://docs.github.com/en/webhooks/using-webhooks/best-practices-for-using-webhooks

• PowerShell on Windows: https://learn.microsoft.com/powershell/scripting/install/installing-powershell-on-windows

Adapted for Windows VS Code and PowerShell on 6 November 2026.

Copyright

© 2026 QA Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.